Deep Generative Models

Detailed Architecture Reference

Autoencoder · Variational Autoencoder · Generative Adversarial Network

AE
Autoencoder

* Encoder + Decoder
* Bottleneck compression
* Reconstruction loss

VAE
Variational AE

* Probabilistic encoding
* Reparameterisation trick
* KL + Reconstruction loss

GAN
Generative Adversarial Net

* Generator vs Discriminator
* Adversarial training
* Minimax loss

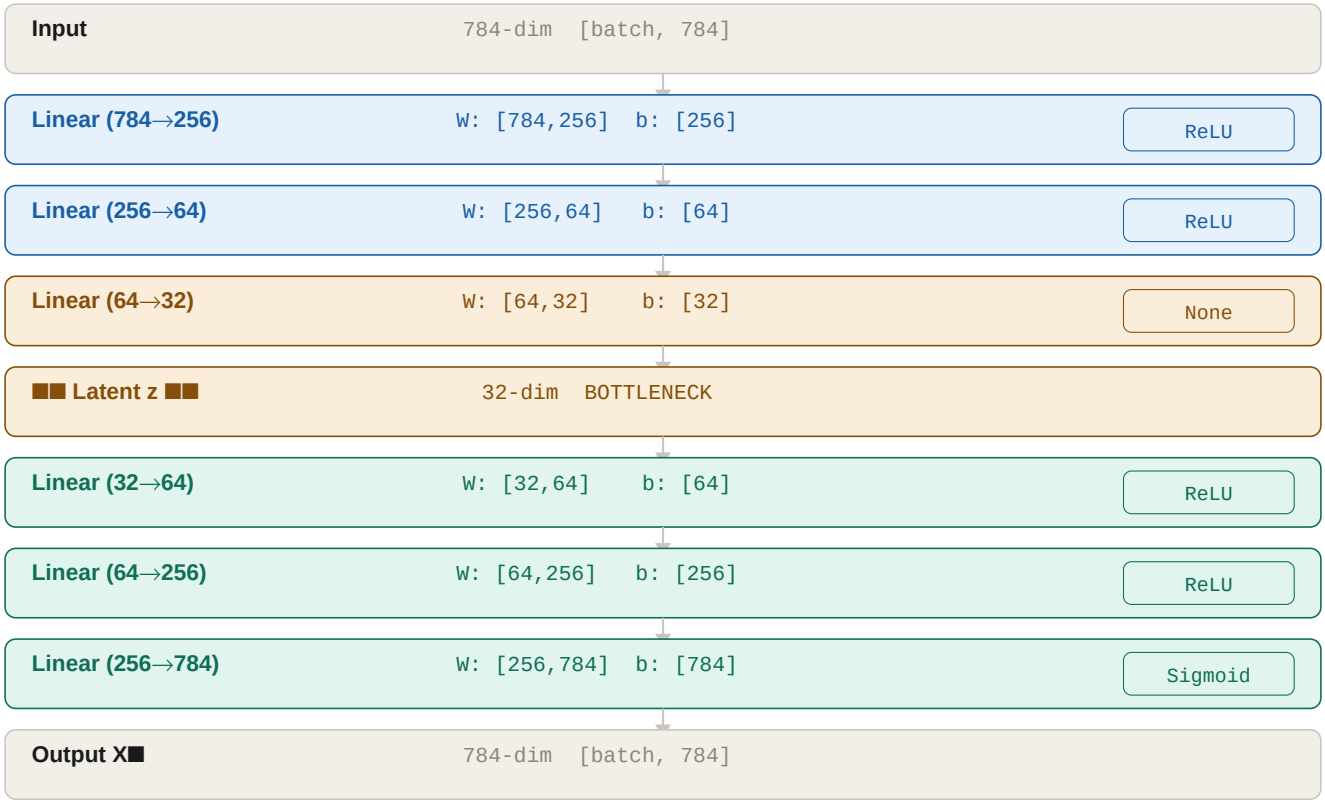
What is inside

+ Layer-by-layer architecture
+ Dimensions at every step
+ Activation functions & why
+ Loss functions explained
+ Architecture diagrams
+ Comparison table

1 · Autoencoder (AE)

An autoencoder is a neural network trained to **compress** high-dimensional input into a compact latent representation, then **reconstruct** it back to the original size. It learns entirely unsupervised — the input itself is the target. The bottleneck forces the network to learn only the most informative features of the data.

Architecture Diagram



Blue = Encoder layers · Amber = Bottleneck · Green = Decoder layers

Layer-by-Layer Detail

Layer / Block	Input Dims	Output Dims	Operation	Activation	Purpose
Input	[B, 784]	[B, 784]	Flatten image	—	Raw pixel values in [0,1]. B = batch size.
Linear 784→256	[B, 784]	[B, 256]	z = Wx + b	ReLU	First compression. Learns low-level features.
Linear 256→64	[B, 256]	[B, 64]	z = Wx + b	ReLU	Deeper compression. Learns abstract features.
Linear 64→32	[B, 64]	[B, 32]	z = Wx + b	None	Bottleneck output. Raw latent code.
Latent z	[B, 32]	[B, 32]	Identity	—	Compressed representation. 32 learned features.
Linear 32→64	[B, 32]	[B, 64]	z = Wx + b	ReLU	Begin reconstruction. Expand features.
Linear 64→256	[B, 64]	[B, 256]	z = Wx + b	ReLU	Further expansion back toward input space.

Layer / Block	Input Dims	Output Dims	Operation	Activation	Purpose
Linear 256→784	[B, 256]	[B, 784]	z = Wx + b	Sigmoid	Final output. Sigmoid squashes values to [0,1].
Output X■	[B, 784]	[B, 784]	Reshape	—	Reconstructed image. Compare with input X.

Mathematical Formulation

Forward pass:

```
h1 = ReLU(W1·x + b1) [784 → 256]
h2 = ReLU(W2·h1 + b2) [256 → 64]
z = W3·h2 + b3 [64 → 32] ← latent code
h3 = ReLU(W4·z + b4) [32 → 64]
h4 = ReLU(W5·h3 + b5) [64 → 256]
x■ = Sigmoid(W6·h4 + b6) [256 → 784] ← output
```

Loss function:

```
L_MSE = (1/n) · Σ(xi − x■i)² ← pixel-wise squared error
L_BCE = −Σ[xi·log(x■i) + (1−xi)·log(1−x■i)] ← treats pixels as Bernoulli
```

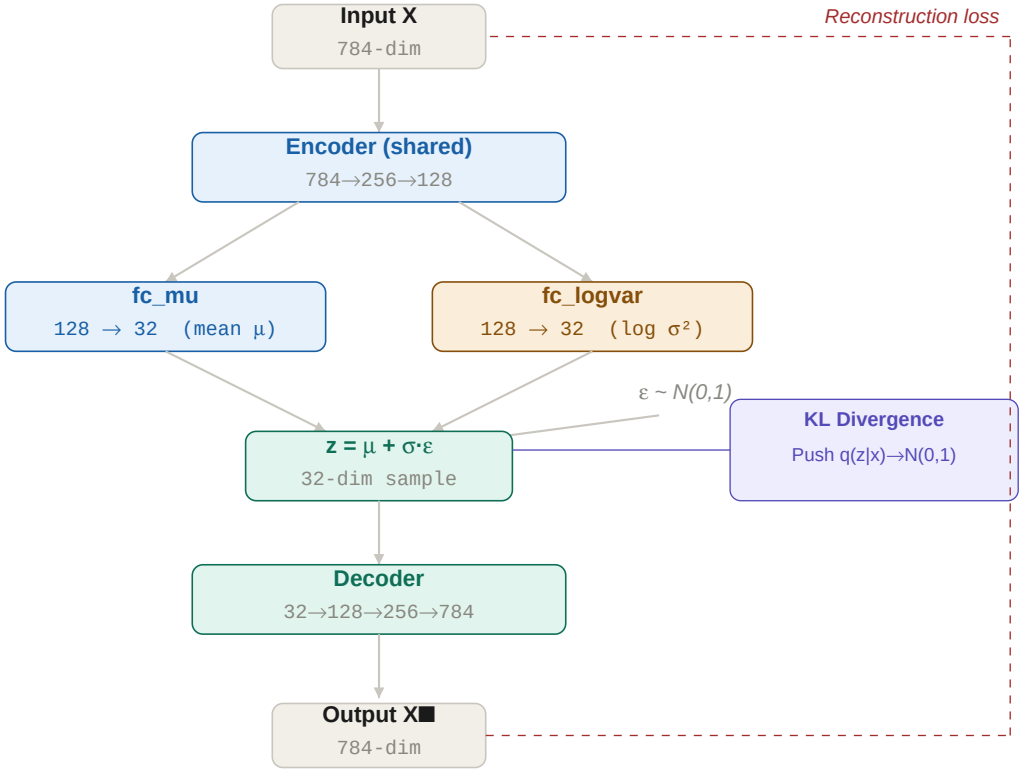
Key Design Decisions

ReLU activation	Used in all hidden layers. Cheap to compute, avoids vanishing gradient. Outputs max(0,x).
Sigmoid on output	Maps output to [0,1] to match normalised pixel range. Match your data normalisation.
No activation on bottleneck	The latent code z has no activation — it can be any real value. This gives the decoder full range.
Symmetric architecture	Decoder mirrors encoder. Not required but common practice — ensures balanced capacity.
Bottleneck size	32 is a hyperparameter. Too small → lossy. Too large → memorisation. Tune for your task.
Loss choice	MSE for continuous data. BCE for binary/pixel data. BCE treats each pixel independently.

2 · Variational Autoencoder (VAE)

A VAE extends the autoencoder by making the latent space **probabilistic**. Instead of encoding each input to a single point, the encoder outputs the **parameters of a Gaussian distribution** (mean μ and log-variance log σ²). We sample the latent code z from this distribution. The KL divergence loss regularises the latent space to follow a standard normal N(0,1), enabling generation of new data by sampling z from N(0,1) and decoding it.

Architecture Diagram (Encoder branches into μ and σ)



The encoder branches into two parallel heads. The reparameterisation trick connects them to the decoder.

Layer-by-Layer Detail

Layer / Block	Input Dims	Output Dims	Operation	Activation	Purpose
Input	[B, 784]	[B, 784]	Flatten	—	Raw pixels. Same as AE input.
Linear 784→256	[B, 784]	[B, 256]	Wx + b	ReLU	Shared encoder: extracts features.
Linear 256→128	[B, 256]	[B, 128]	Wx + b	ReLU	Shared encoder: deeper abstraction.
fc_mu (128→32)	[B, 128]	[B, 32]	Wx + b	None	HEAD 1: outputs mean vector μ.
fc_logvar (128→32)	[B, 128]	[B, 32]	Wx + b	None	HEAD 2: outputs log σ². No activation (can be any real).
Reparameterise	μ, logvar	[B, 32]	μ + σ·ε	N(0,1)	σ=exp(0.5·logvar). ε~N(0,1). Differentiable sampling.

Layer / Block	Input Dims	Output Dims	Operation	Activation	Purpose
Latent z	[B, 32]	[B, 32]	Sample	—	Stochastic latent code. Different each forward pass.
Linear 32→128	[B, 32]	[B, 128]	Wx + b	ReLU	Decoder: begin reconstruction.
Linear 128→256	[B, 128]	[B, 256]	Wx + b	ReLU	Decoder: expand representation.
Linear 256→784	[B, 256]	[B, 784]	Wx + b	Sigmoid	Decoder output: reconstructed image.
Output X■	[B, 784]	[B, 784]	Reshape	—	Compared to input for reconstruction loss.

Mathematical Formulation

Encoder (probabilistic):

```
h = ReLU(W2·ReLU(W1·x + b1) + b2) [784 → 128]
μ = W_mu·h + b_mu [128 → 32] ← mean
log σ² = W_lv·h + b_lv [128 → 32] ← log variance
```

Reparameterisation trick:

```
σ = exp(0.5 · log σ²) ← std deviation (always positive)
ε ~ N(0, I) ← random noise (not learned)
z = μ + σ · ε ← differentiable sample
```

Loss function (ELBO — Evidence Lower Bound):

```
L_recon = −Σ[x·log(x■) + (1−x)·log(1−x■)] ← reconstruction (BCE)
L_KL = −½ · Σ(1 + log σ² − μ² − σ²) ← KL divergence from N(0,1)
L_total = L_recon + β · L_KL ← β=1 (standard VAE)
```

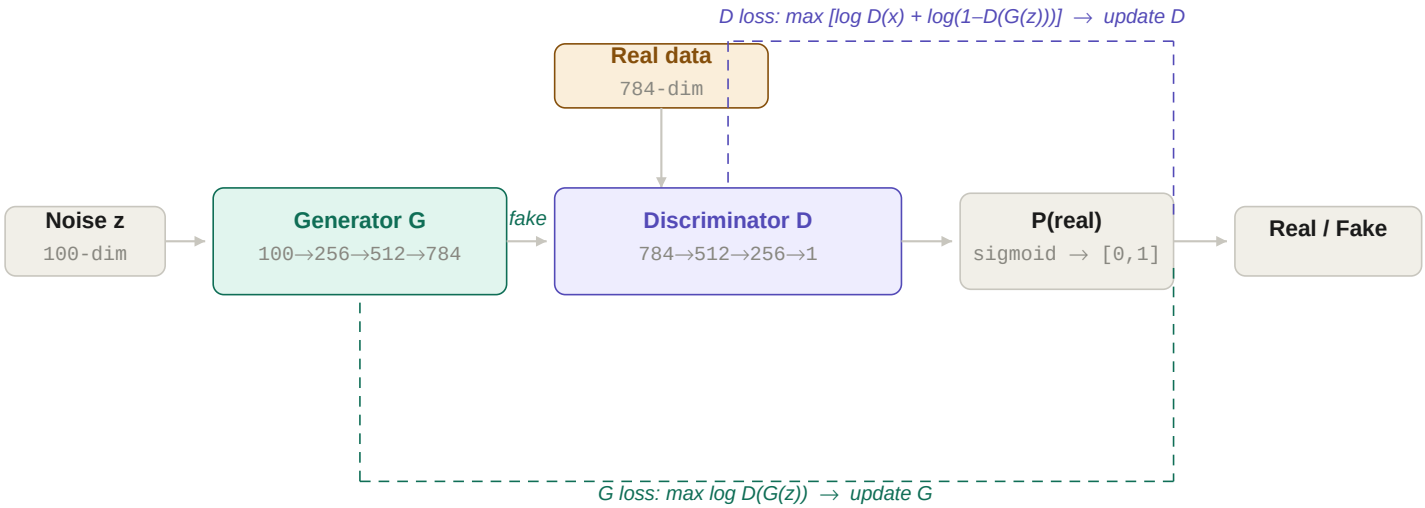
Key Design Decisions

Two encoder heads	fc_mu and fc_logvar share the same upstream layers but have separate weights. This lets them specialise independently.
Log variance not σ	We predict log σ² (not σ) because log σ² is unbounded. σ must be positive; log σ² can be any real — easier to optimise.
Reparameterisation trick	Makes random sampling differentiable. ε is injected as fixed noise; gradients flow through μ and σ only.
KL divergence purpose	Without KL, the encoder can set σ→0 and collapse to a plain AE. KL forces the space to remain a smooth, explorable distribution.
β hyperparameter	β>1 (β-VAE) produces a more disentangled latent space at the cost of reconstruction quality. Common in research.
Generation at inference	Skip the encoder. Sample z~N(0,1) and decode. Works because KL forces q(z|x) to match p(z)=N(0,1).

3 · Generative Adversarial Network (GAN)

A GAN consists of two adversarially trained networks: a **Generator G** that maps random noise to synthetic data, and a **Discriminator D** that classifies data as real or fake. G is trained to fool D; D is trained to catch G. They alternate updates — neither has direct access to the other's gradients. This competition drives both toward Nash equilibrium where G produces indistinguishable fakes.

Architecture Diagram



Green = Generator path · Purple = Discriminator path · Dashed = gradient feedback

Generator G — Layer-by-Layer

Layer / Block	Input Dims	Output Dims	Operation	Activation	Purpose
Noise input z	[B, 100]	[B, 100]	Sample N(0,1)	—	Pure random noise. Seed for generation.
Linear 100→256	[B, 100]	[B, 256]	Wx + b	ReLU	Begin expanding noise into structure.
Linear 256→512	[B, 256]	[B, 512]	Wx + b	ReLU	More capacity to build complex patterns.
Linear 512→784	[B, 512]	[B, 784]	Wx + b	Tanh	Output in [–1,1]. Data normalised to match.
Fake image	[B, 784]	[B, 784]	Reshape	—	Generated fake. Sent to discriminator.

Discriminator D — Layer-by-Layer

Layer / Block	Input Dims	Output Dims	Operation	Activation	Purpose
Input (real/fake)	[B, 784]	[B, 784]	Flatten	—	Receives both real and fake images.
Linear 784→512	[B, 784]	[B, 512]	Wx + b	LeakyReLU(0.2)	LeakyReLU prevents dead neurons. Slope=0.2 below zero.
Linear 512→256	[B, 512]	[B, 256]	Wx + b	LeakyReLU(0.2)	Deeper feature extraction for discrimination.

Layer / Block	Input Dims	Output Dims	Operation	Activation	Purpose
Linear 256→1	[B, 256]	[B, 1]	Wx + b	Sigmoid	Single output probability: P(input is real).
Output P(real)	[B, 1]	[B, 1]	Threshold	—	≈1.0 = real, ≈0.0 = fake.

Mathematical Formulation

Minimax objective:

```
min_G max_D V(G,D) = E[log D(x)] + E[log(1 − D(G(z)))]
```

Discriminator loss (maximise ability to classify):

```
L_D = −E[log D(x)] − E[log(1 − D(G(z)))] ← minimise this
= BCE(D(real), 1) + BCE(D(fake), 0) ← practical form
```

Generator loss (non-saturating, practical version):

```
L_G = −E[log D(G(z))] ← maximise D(G(z))
= BCE(D(G(z)), 1) ← fool D into saying "real"
```

Training alternation (each batch):

```
Step 1: freeze G → compute L_D → backprop → update D
Step 2: freeze D → compute L_G → backprop → update G
```

Key Design Decisions

Tanh on G output	Generator output in [−1,1]. Requires normalising real training images to [−1,1] (not [0,1]) to match.
LeakyReLU in D	Standard ReLU creates dead neurons (zero gradient for negative inputs). LeakyReLU slope=0.2 keeps gradients flowing.
.detach() on fake	When training D, call G(z).detach(). Without it, gradients flow into G during D's update, corrupting G's weights.
Non-saturating G loss	Original: min log(1−D(G(z))). Problem: saturates early (gradient≈0). Better: max log(D(G(z))) = BCE(D(fake), 1).
Adam betas=(0.5,0.999)	Default β1=0.9 causes oscillation. β1=0.5 reduces momentum, stabilising adversarial training. DCGAN finding.
Noise z dimension	100 is conventional. Larger z = more diversity but harder training. Smaller z = faster but less variety.

4 · Architecture Comparison

Aspect	Autoencoder (AE)	Variational AE (VAE)	GAN
Input to model	Image x	Image x	Noise z ~ N(0,1)
Encoder	Yes — maps x → z	Yes — maps x → (μ, σ)	No encoder
Latent space	Fixed point per input	Gaussian distribution per input	Pure noise vector
Decoder / Generator	Maps z → x■	Maps sampled z → x■	Maps z → fake image
Second network	None	None	Discriminator D
Loss function	Reconstruction (MSE/BCE)	Reconstruction + KL divergence	Adversarial BCE
Can generate new data	No (unstructured space)	Yes (sample z ~ N(0,1))	Yes (sample z ~ N(0,1))
Output quality	Accurate but no generation	Smooth but blurry	Sharp and realistic
Training stability	Very stable	Stable	Unstable — needs tuning
Activation (output)	Sigmoid → [0,1]	Sigmoid → [0,1]	Tanh → [–1,1]
Hidden activations	ReLU	ReLU	ReLU (G), LeakyReLU (D)
Latent dim (typical)	32–128	32–128	100
Key hyperparameter	Bottleneck size	β (KL weight)	Learning rate ratio G/D
Main use case	Compression, denoising	Generation, interpolation	Image synthesis, deepfakes
Training labels	None (unsupervised)	None (unsupervised)	Real/Fake (self-labelled)

When to use which model

Task	Recommended	Reason
Anomaly detection	Autoencoder	High reconstruction error = anomaly. No generation needed.
Feature extraction for downstream ML	Autoencoder	Encoder produces compact, informative features.
Smooth interpolation between samples	VAE	Continuous latent space allows meaningful interpolation.
Synthetic tabular data (balanced classes)	VAE	Stable training, structured generation. Good for imbalanced datasets.
Photorealistic image generation	GAN	Adversarial loss produces sharp, high-quality outputs.
Data augmentation (images)	GAN	Generate diverse realistic variants of training data.